第 1 章　绪论

1．简述下列概念：数据、数据元素、数据项、数据对象、数据结构、逻辑结构、存储结构、抽象数据类型。

答案：

数据：是客观事物的符号表示，指所有能输入到计算机中并被计算机程序处理的符号的总称。如数学计算中用到的整数和实数，文本编辑所用到的字符串，多媒体程序处理的图形、图像、声音、动画等通过特殊编码定义后的数据。

数据元素：是数据的基本单位，在计算机中通常作为一个整体进行考虑和处理。在有些情况下，数据元素也称为元素、结点、记录等。数据元素用于完整地描述一个对象，如一个学生记录，树中棋盘的一个格局（状态）、图中的一个顶点等。

数据项：是组成数据元素的、有独立含义的、不可分割的最小单位。例如，学生基本信息表中的学号、姓名、性别等都是数据项。

数据对象：是性质相同的数据元素的集合，是数据的一个子集。例如：整数数据对象是集合 N={0，±1，±2，…}，字母字符数据对象是集合 C={‘A’，‘B’，…，‘Z’，‘a’，‘b’，…，‘z’}，学生基本信息表也可是一个数据对象。

数据结构：是相互之间存在一种或多种特定关系的数据元素的集合。换句话说，数据结构是带“结构”的数据元素的集合，“结构”就是指数据元素之间存在的关系。

逻辑结构：从逻辑关系上描述数据，它与数据的存储无关，是独立于计算机的。因此，数据的逻辑结构可以看作是从具体问题抽象出来的数学模型。

存储结构：数据对象在计算机中的存储表示，也称为**物理结构**。

抽象数据类型：由用户定义的，表示应用问题的数学模型，以及定义在这个模型上的一组操作的总称。具体包括三部分：数据对象、数据对象上关系的集合和对数据对象的基本操作的集合。

2．简述逻辑结构的四种基本关系并画出它们的关系图。

答案：

（1）集合结构

数据元素之间除了“属于同一集合”的关系外，别无其他关系。例如，确定一名学生是否为班级成员，只需将班级看做一个集合结构。

（2）线性结构

数据元素之间存在一对一的关系。例如，将学生信息数据按照其入学报到的时间先后顺序进行排列，将组成一个线性结构。

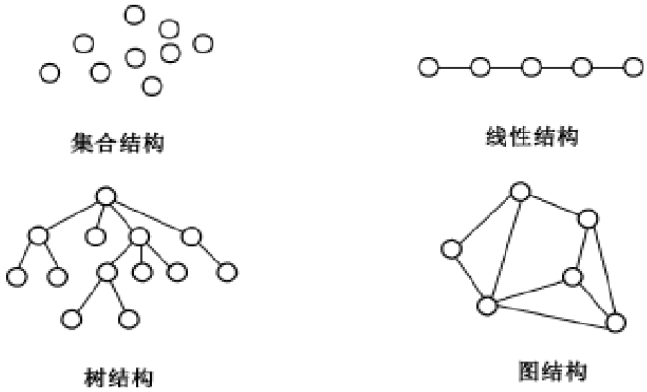
（3）树结构

数据元素之间存在一对多的关系。例如，在班级的管理体系中，班长管理多个组长，每位组长管理多名组员，从而构成树形结构。

（4）图结构或网状结构

数据元素之间存在多对多的关系。例如，多位同学之间的朋友关系，任何两位同学都可以是朋友，从而构成图形结构或网状结构。

其中树结构和图结构都属于非线性结构。



四类基本逻辑结构关系图

3．存储结构由哪两种基本的存储方法实现？

答案：

（1）顺序存储结构

顺序存储结构是借助元素在存储器中的相对位置来表示数据元素之间的逻辑关系，通常借助程序设计语言的数组类型来描述。

（2）链式存储结构

顺序存储结构要求所有的元素依次存放在一片连续的存储空间中，而链式存储结构，无需占用一整块存储空间。但为了表示结点之间的关系，需要给每个结点附加指针字段，用于存放后继元素的存储地址。所以链式存储结构通常借助于程序设计语言的指针类型来描述。

4．选择题

C C D D

5．试分析下面各程序段的时间复杂度。

（1）

O(1)

（2）

答案：O(m*n)

（3）答案：$O(n^2)$

（4）

答案：$O(\log_3 n)$

解释：语句 i=i*3;的执行次数为$\lfloor \log_3 n \rfloor$。

（5）

答案：$O(n^2)$

解释：语句 x++;的执行次数为 n-1+n-2+……＋1= n(n-1)/2。

第 2 章　线性表

1．选择题

（1）答案：B

解释：顺序表中的数据连续存储，所以第 5 个元素的地址为：100+2*4=108。

（2）答案：A

解释：在顺序表中插入一个结点的时间复杂度都是 $O(n^2)$，排序的时间复杂度为 $O(n^2)$ 或 $O(n\log_2 n)$。顺序表是一种随机存取结构，访问第 i 个结点和求第 i 个结点的直接前驱都可以直接通过数组的下标直接定位，时间复杂度是 O(1)。

（3）

答案：B

解释：平均要移动的元素个数为：n/2。

（4）

答案：A

（5）

答案：D

（6）

答案：B

解释：链表最大的优点在于插入和删除时不需要移动数据，直接修改指针即可。

（7）

答案：C

解释：存储密度是指一个结点数据本身所占的存储空间和整个结点所占的存储空间之比，假设单链表一个结点本身所占的空间为 D，指针域所占的空间为 N，则存储密度为：D/(D+N)，一定小于 1。

（8）

答案：A

解释：当第一个有序表中所有的元素都小于（或大于）第二个表中的元素，只需要用第二个表中的第一个元素依次与第一个表的元素比较，总计比较 n 次。

（9）

答案：B

(10)

答案：D

(11)

答案：C

解释：单链表创建的时间复杂度是 O(n)，而要建立一个有序的单链表，则每生成一个新结点时需要和已有的结点进行比较，确定合适的插入位置，所以时间复杂度是

O(n2)。

(12)

答案：D

解释：链式存储结构和顺序存储结构各有优缺点，有不同的适用场合。

(13)

答案：D

(14)

答案：A

(15)

答案：C

2．算法设计题

（1）已知一个顺序表中的各元素值是从小到大有序的，设计一个算法，插入一个值为 x 的数据元素使其仍然是从小到大有序。（类似于有序表的合并，大家思考）

（2）

[题目分析]

合并后的新表使用头指针 Lc 指向，pa 和 pb 分别是链表 La 和 Lb 的工作指针,初始化为相应链表的第一个结点，从第一个结点开始进行比较，当两个链表 La 和 Lb 均为到达表尾结点时，依次摘取其中较小者重新链接在 Lc 表的最后。如果两个表中的元素相等，只摘取 La 表中的元素，删除 Lb 表中的元素，这样确保合并后表中无重复的元素。当一个表到达表尾结点，为空时，将非空表的剩余元素直接链接在 Lc 表的最后。

[算法描述]

```
void MergeList(LinkList &La,LinkList &Lb,LinkList &Lc)
{//合并链表 La 和 Lb，合并后的新表使用头指针 Lc 指向
  pa=La->next;  pb=Lb->next;
   //pa 和 pb 分别是链表 La 和 Lb 的工作指针,初始化为相应链表的第一个结点
   Lc=pc=La;  //用 La 的头结点作为 Lc 的头结点
   while(pa && pb)
   {if(pa->data<pb->data){pc->next=pa;pc=pa;pa=pa->next;}
     //取较小者 La 中的元素，将 pa 链接在 pc 的后面，pa 指针后移
     else if(pa->data>pb->data) {pc->next=pb; pc=pb; pb=pb->next;}
      //取较小者 Lb 中的元素，将 pb 链接在 pc 的后面，pb 指针后移
     else //相等时取 La 中的元素，删除 Lb 中的元素
     {pc->next=pa;pc=pa;pa=pa->next;
      q=pb->next;delete pb ;pb =q;
      }
     }
    pc->next=pa?pa:pb;     //插入剩余段
     delete Lb;             //释放 Lb 的头结点
```

```
}
```

（3）

[题目分析]

假定第一个结点中数据具有最大值，依次与下一个元素比较，若其小于下一个元素，则设其下一个元素为最大值，反复进行比较，直到遍历完该链表。

[算法描述]

```
ElemType Max (LinkList L ){
    if(L->next==NULL) return NULL;
    pmax=L->next; //假定第一个结点中数据具有最大值
    p=L->next->next;
    while(p != NULL ){//如果下一个结点存在
        if(p->data > pmax->data) pmax=p;//如果 p 的值大于 pmax 的值，则重新赋值
        p=p->next;//遍历链表
    }
    return pmax->data;
```

第 3 章　栈和队列

1．选择题

（1）

答案：C

解释：栈是后进先出的线性表，不难发现 C 选项中元素 1 比元素 2 先出栈，违背了栈的后进先出原则，所以不可能出现 C 选项所示的情况。

（2）

答案：C

解释：栈是后进先出的线性表，一个栈的入栈序列是 1，2，3，…，n，而输出序列的第一个元素为 n，说明 1，2，3，…，n 一次性全部进栈，再进行输出，所以 p1=n，p2=n-1，…，pi=n-i+1。

（3）

答案：D

解释：对于非循环队列，尾指针和头指针的差值便是队列的长度，而对于循环队列，差值可能为负数，所以需要将差值加上 MAXSIZE（本题为 n），然后与 MAXSIZE（本题为 n）求余，即（n+r-f)%n。

（4）

答案：A

解释：x=top->data 将结点的值保存到 x 中，top=top->link 栈顶指针指向栈顶下一结点，即摘除栈顶结点。

（5）

答案：A

解释：解决缓冲区问题应利用一种先进先出的线性表，而队列正是一种先进先出的线性表。

（6）

答案：B

解释：元素出队的序列是 e2、e4、e3、e6、e5 和 e1，可知元素入队的序列是 e2、e4、e3、e6、e5 和 e1，即元素出栈的序列也是 e2、e4、e3、e6、e5 和 e1，而元素 e1、e2、e3、e4、e5 和 e6 依次进入栈，易知栈 S 中最多同时存在 3 个元素，故栈 S 的容量至少为 3。

（7）

答案：C

解释：初始栈顶指针 top 为 n+1，说明元素从数组向量的高端地址进栈，又因为元素存储在向量空间 V[1..n]中，所以进栈时 top 指针先下移变为 n，之后将元素 x 存储在 V[n]。

（8）

答案：D

解释：一般情况下只修改头指针，但是，当删除的是队列中最后一个元素时，队尾指针也丢失了，因此需对队尾指针重新赋值。

（9）

答案：D

解释：数组 A[0..m]中共含有 m+1 个元素，故在求模运算时应除以 m+1。

（10）

答案：B

解释：最大容量为 n 的循环队列，队满条件是(rear+1)%n==front，队空条件是 rear==front。

（11）

答案：C

解释：栈只允许在栈顶处进行插入和删除元素，队列只允许在队尾插入元素和在队头删除元素。

2．算法设计题

（1）从键盘上输入一个后缀表达式，试编写算法计算表达式的值。规定：逆波兰表达式的长度不超过一行，以$符作为输入结束，操作数之间用空格分隔,操作符只可能有+、-、*、/四种运算。例如：234 34+2*$。

[题目分析]

逆波兰表达式(即后缀表达式)求值规则如下：设立运算数栈 OPND,对表达式从左到右扫描(读入)，当表达式中扫描到数时，压入 OPND 栈。当扫描到运算符时，从 OPND 退出两个数，进行相应运算，结果再压入 OPND 栈。这个过程一直进行到读出表达式结束符$，这时 OPND 栈中只有一个数，就是结果。

[算法描述]

```
float expr( )
//从键盘输入逆波兰表达式，以‘$’表示输入结束，本算法求逆波兰式表达式的值。
{float OPND[30];   // OPND 是操作数栈。
 init(OPND);       //两栈初始化。
 float num=0.0;    //数字初始化。
 cin>>x;//x 是字符型变量。
 while(x!=’$’)
    {switch
      {case ‘0’<=x<=’9’:
           while((x>=’0’&&x<=’9’)||x==’.’)  //拼数
               if(x!=’.’)   //处理整数
                   {num=num*10+(ord(x)-ord(‘0’)); cin>>x;}
               else          //处理小数部分。
```

```
                {scale=10.0; cin>>x;
                while(x>=' 0' &&x<=' 9' )
                {num=num+(ord(x)-ord( '0' )/scale;
                scale=scale*10;  cin>>x; }
                }//else
            push(OPND,num); num=0.0;//数压入栈，下个数初始化
        case x=' ' :break;  //遇空格，继续读下一个字符。
        case x='+' :push(OPND,pop(OPND)+pop(OPND));break;
        case x='-' :x1=pop(OPND);x2=pop(OPND);push(OPND,x2-x1);break;
        case x='*' :push(OPND,pop(OPND)*pop(OPND));break;
        case x='/' :x1=pop(OPND);x2=pop(OPND);push(OPND,x2/x1);break;
        default:        //其它符号不作处理。
      }//结束 switch
      cin>>x;//读入表达式中下一个字符。
    }//结束 while（x! ='$' ）
    cout<<"后缀表达式的值为"<<pop(OPND);
  }//算法结束。
```

[算法讨论]假设输入的后缀表达式是正确的，未作错误检查。算法中拼数部分是核心。若遇到大于等于‘0’且小于等于‘9’的字符，认为是数。这种字符的序号减去字符‘0’的序号得出数。对于整数，每读入一个数字字符，前面得到的部分数要乘上 10 再加新读入的数得到新的部分数。当读到小数点，认为数的整数部分已完，要接着处理小数部分。小数部分的数要除以 10（或 10 的幂数）变成十分位，百分位，千分位数等等，与前面部分数相加。在拼数过程中，若遇非数字字符，表示数已拼完，将数压入栈中，并且将变量 num 恢复为 0，准备下一个数。这时对新读入的字符进入‘+’、‘-’、‘*’、‘/’及空格的判断，因此在结束处理数字字符的 case 后，不能加入 break 语句。

第 4 章　串、数组和广义表

1．选择题

（1）

答案：B

（2）

答案：B

解释：空格常常是串的字符集合中的一个元素，有一个或多个空格组成的串成为空格串，零个字符的串成为空串，其长度为零。

（3）

答案：B

解释：串中字符的数目称为串的长度。

第5章 数组和广义表

（1）

答案：B

解释：以行序为主，则 LOC[5,5]=[（5-1）*100+（5-1）]*2+10=818。

（2）

答案：B

解释：以列序为主，则 LOC[5,8]=[（8-1）*8+（5-1）]*3+BA=BA+180。

（3）

答案：C

（4）

答案：B

（5）

答案：B

解释：设数组从内存首地址 M 开始顺序存放，若数组按行先存储，元素 A[8,5]的起始地址为：M+[（8-0）*10+（5-1）]*1=M+84；若数组按列先存储，易计算出元素 A[3,10]的起始地址为：M+[（10-1）*9+（3-0）]*1=M+84。故选 B。

（6）

答案：B

解释：共有 5*3*3=45 个元素。

（7）

答案：D

解释：Tail(A)=(b,(c,d),(e,(f,g)))；Tail(Tail(A))=((c,d),(e,(f,g)))； Head(Tail(Tail(A)))=(c,d)；Tail(Head(Tail(Tail(A))))=(d)；Head(Tail(Head(Tail(Tail(A)))))=d。

（8）

答案：C、B

解释：表头为非空广义表的第一个元素，可以是一个单原子，也可以是一个子表，((a,b,c,d))的表头为一个子表(a,b,c,d)；表尾为除去表头之外，由其余元素构成的表，表为一定是个广义表，((a,b,c,d))的表尾为空表( )。

2．应用题

（1）

答案：

每个元素 32 个二进制位，主存字长 16 位，故每个元素占 2 个字长，行下标可平移至 1 到 11。

（1）242　（2）22　（3）s+182　（4）s+142

(2)

答案：H（H（T（H（T（H（T（L））））））

第 6 章　树和二叉树

1．选择题

（1）

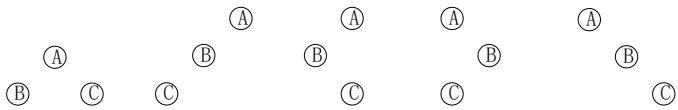
答案：A

解释：因为二叉树有左孩子、右孩子之分，故一棵树转换为二叉树后，这棵二叉树的形态是唯一的。

（2）

答案：D

解释：五种情况如下：



（3）

答案：D

解释：设度为 0 结点（叶子结点）个数为 A，度为 1 的结点个数为 B，度为 2 的结点个数为 C，有 A=C+1，A+B+C=1001，可得 2C+B=1000，由完全二叉树的性质可得 B=0 或 1，又因为 C 为整数，所以 B=0，C=500，A=501，即有 501 个叶子结点。

（4）

答案：C

解释：若每层仅有一个结点，则树高 h 为 1025；且其最小树高为$\lfloor \log_2 1025 \rfloor + 1 = 11$，即 h 在 11 至 1025 之间。

（5）

答案：A

解释：深度为 h 的满 m 叉树共有 m^h-1 个结点，第 k 层有 m^{k-1} 个结点。

（6）

答案：C

解释：利用二叉链表存储树时，右指针指向兄弟结点，因为根节点没有兄弟结点，故根节点的右指针指向空。

（7）

答案：C

解释：根据题意可知按照先左孩子、再右孩子、最后双亲结点的顺序遍历二叉树，即后序遍历二叉树。

（8）

答案：C

解释：后续遍历和层次遍历均可实现左右子树的交换，不过层次遍历的实现消耗比后续大，后序遍历方法最合适。

（9）

答案：D

解释：树的存储结构有三种：双亲表示法、孩子表示法、孩子兄弟表示法，其中孩子兄弟表示法是常用的表示法，任意一棵树都能通过孩子兄弟表示法转换为二叉树进行存储。

（10）

答案：C

解释：因为先序遍历结果是“中左右”，后序遍历结果是“左右中”，当没有左子树时，就是“中右”和“右中”；当没有右子树时，就是“中左”和“左中”。则所有的结点均无左孩子或所有的结点均无右孩子均可，所以 A、B 不能选，又所有的结点均无左孩子与所有的结点均无右孩子时，均只有一个叶子结点，故选 C。

（11）

答案：B

解释：在哈夫曼树中没有度为 1 的结点，只有度为 0（叶子结点）和度为 2 的结点。设叶子结点的个数为 n0，度为 2 的结点的个数为 n2，由二叉树的性质 n0=n2+1，则总结点数 n=n0+n2=2*n0-1，得到 n0=100。

（12）

答案：C

（13）

答案：A

（14）

答案：C

（15）

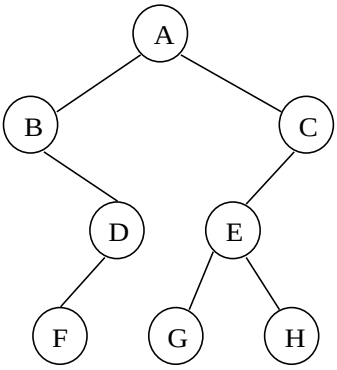
答案：A

解释：哈夫曼树的构造过程是每次都选取权值最小的树作为左右子树构造一棵新的二叉树，所以树中一定没有度为 1 的结点、两个权值最小的结点一定是兄弟结点、任一非叶结点的权值一定不小于下一层任一结点的权值。

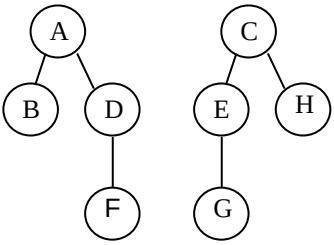
2．应用题

（1）

答案：



①

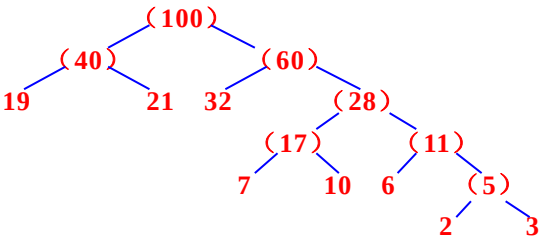


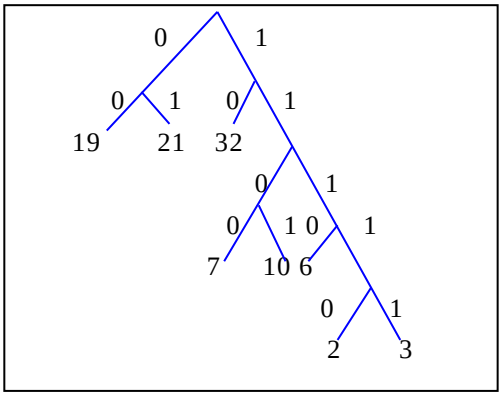
②

（2）答案：方案 1；哈夫曼编码

先将概率放大 100 倍，以方便构造哈夫曼树。

w={7,19,2,6,32,3,21,10}，按哈夫曼规则：【[（2,3），6], (7,10)】，……19, 21, 32





方案比较：

字母编号	对应编码	出现频率
1	1100	0.07
2	00	0.19
3	11110	0.02
4	1110	0.06
5	10	0.32
6	11111	0.03
7	01	0.21
8	1101	0.10

字母编号	对应编码	出现频率
1	000	0.07
2	001	0.19
3	010	0.02
4	011	0.06
5	100	0.32
6	101	0.03
7	110	0.21
8	111	0.10

方案 1 的 WPL＝2(0.19+0.32+0.21)+4(0.07+0.06+0.10)+5(0.02+0.03)=1.44+0.92+0.25=2.61

方案 2 的 WPL＝3(0.19+0.32+0.21+0.07+0.06+0.10+0.02+0.03)=3

结论：哈夫曼编码优于等长二进制编码

第 6 章 图

1．选择题

（1）

答案：C

（2）

答案：B

解释：有向图所有顶点入度之和等于所有顶点出度之和。

（3）

答案：B

解释：有向图的边有方向之分，即为从 n 个顶点中选取 2 个顶点有序排列，结果为 n(n-1)。

（4）

答案：B

（5）

答案：C

解释：8 个顶点的无向图最多有 8*7/2=28 条边，再添加一个点即构成非连通无向图，故至少有 9 个顶点。

（6）

答案：B

解释：即从该无向图任意一个顶点出发有到各个顶点的路径，所以该无向图是连通图。

（7）

答案：A

解释：Prim 算法适合构造一个稠密图 G 的最小生成树，Kruskal 算法适合构造一个稀疏图 G 的最小生成树。

（8）

答案：B

解释：广度优先遍历通常借助队列来实现算法，深度优先遍历通常借助栈来实现算法。

（9）

答案：A

解释：广度优先遍历通常借助队列来实现算法，深度优先遍历通常借助栈来实现算法。

（10）

答案：A

（11）

答案：D

（12）答案：c

（13）

答案：D、D

（14）

答案：B

2．应用题

（1）

答案：

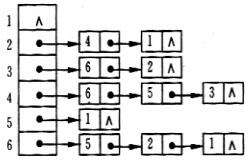
(1)

顶点	1	2	3	4	5	6
入度	3	2	1	1	2	2
出度	0	2	2	3	1	3

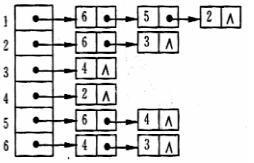
(2) 邻接矩阵

$$\begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 & 1 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 1 & 0 \end{bmatrix}$$

(3) 邻接表



(4) 逆邻接表



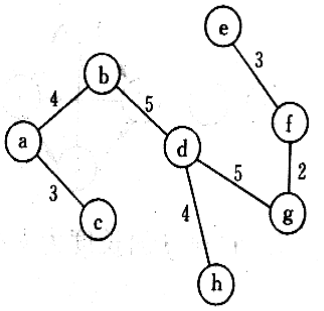
（2）

答案：

①

$$\begin{bmatrix} \infty & 4 & 3 & \infty & \infty & \infty & \infty & \infty \\ 4 & \infty & 5 & 5 & 9 & \infty & \infty & \infty \\ 3 & 5 & \infty & 5 & \infty & \infty & \infty & 5 \\ \infty & 5 & 5 & \infty & 7 & 6 & 5 & 4 \\ \infty & 9 & \infty & 7 & \infty & 3 & \infty & \infty \\ \infty & \infty & \infty & 6 & 3 & \infty & 2 & \infty \\ \infty & \infty & \infty & 5 & \infty & 2 & \infty & 6 \\ \infty & \infty & 5 & 4 & \infty & \infty & 6 & \infty \end{bmatrix}$$

③



②

a	→	b	4	→	c	3					
b	→	a	4	→	c	5	→	d	5	→	e	9
c	→	a	3	→	b	5	→	d	5	→	h	5

d	→	b	5	→	c	5	→	e	7	→	f	6	→	g	5	→	h	4
e	→	b	9	→	d	7	→	f	3								
f	→	d	6	→	e	3	→	g	2								
g	→	d	5	→	f	2	→	h	6								

（3）

D 终点	i=1	i=2	i=3	i=4	i=5	i=6
b	15 (a,b)	15 (a,b)	15 (a,b)	15 (a,b)	15 (a,b)	**15 (a,b)**
c	**2 (a,c)**					
d	12 (a,d)	12 (a,d)	11 (a,c,f,d)	**11 (a,c,f,d)**		
e	∞	10 (a,c,e)	**10 (a,c,e)**			
f	∞	**6 (a,c,f)**				
g	∞	∞	16 (a,c,f,g)	16 (a,c,f,g)	**14 (a,c,f,d,g)**	
S 终点集	{a,c}	{a,c,f}	{a,c,f,e}	{a,c,f,e,d}	{a,c,f,e,d,g}	{a,c,f,e,d,g,b}

加红部分为路径

（5）

答案：按拓扑有序的顺序计算各个顶点的最早可能开始时间 *Ve* 和最迟允许开始时间 *Vl*。然后再计算各个活动的最早可能开始时间 e 和最迟允许开始时间 *l*，根据 *l* - *e* = 0? 来确定关键活动，从而确定关键路径。

	1 ∂	2 ÷	3 •	4 ≠	5 ≡	6 ≈
Ve	0	19	15	29	38	43

Vl	0	19	15	37	38	43

	<1, 2>	<1, 3>	<3, 2>	<2, 4>	<2, 5>	<3, 5>	<4, 6>	<5, 6>
e	0	0	15	19	19	15	29	38
l	17	0	15	27	19	27	37	38
-e	17	0	0	8	0	12	8	0

此工程最早完成时间为 43。关键路径为<1, 3><3, 2><2, 5><5, 6>

第 7 章　查找

1．选择题

（1）

答案：C

解释：总查找次数 N=1+2+3+…+n=n(n+1)/2，则平均查找长度为 N/n=(n+1)/2。

（2）

答案：D

解释：折半查找要求线性表必须采用顺序存储结构，而且表中元素按关键字有序排列。

（3）

答案：C

解释：分块查找的优点是：在表中插入和删除数据元素时，只要找到该元素对应的块，就可以在该块内进行插入和删除运算。由于块内是无序的，故插入和删除比较容易，无需进行大量移动。如果线性表既要快速查找又经常动态变化，则可采用分块查找。

（4）

答案：A

解释：表中共 10 个元素，第一次取$\lfloor(1+10)/2\rfloor$=5，与第五个元素 20 比较，58 大于 20，再取$\lfloor(6+10)/2\rfloor$=8，与第八个元素 70 比较，依次类推再与 30、50 比较，最终查找失败。

（5）

答案：B

解释：22 个记录的有序表，其折半查找的判定树深度为$\lfloor\log_2 22\rfloor + 1$=5，且该判定树不是满二叉树，即查找失败时至多比较 5 次，至少比较 4 次。

（6）

答案：C

解释：A、B、C、D 四个选项构造二叉排序树都以 100 为根，易知 A、B、D 三个序列中第一个比 100 小的关键字为 80，即 100 的左孩子为 80，而 C 选项中 100 的左孩子为 60，故选 C。

（7）

答案：C

（8）

答案：C

（9）

答案：A

解释：在同义词构成的单链表中，查找该单链表表中不同元素，所消耗的时间不同。

（10）

答案：D

解释：关键字 15 放入位置 4，关键字 38 放入位置 5，关键字 61 放入位置 6，关键字 84

放入位置 7，再添加关键字 49，计算得到地址为 5，冲突，用二次探测法解决冲突得到新地址为 6，仍冲突，再用用二次探测法解决冲突，得到新地址为 4，仍冲突，再用用二次探测法解决冲突，得到新地址为 9，不冲突，即将关键字 49 放入位置 9。

（11）

答案：A

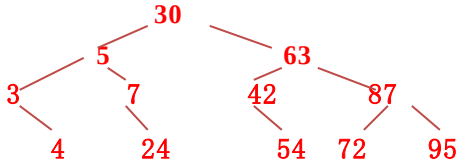
解释：所探测的这些关键字可能是在处理其它关键字冲突过程中放入该位置的。

2．应用题

（1）

答案：

①先画出判定树如下（注：mid=⌊(1+12)/2⌋=6）：



②查找元素 54，需依次与 30, 63, 42, 54 元素比较；

③查找元素 90，需依次与 30, 63,87, 95 元素比较；

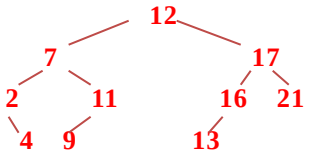
④求 ASL 之前，需要统计每个元素的查找次数。判定树的前 3 层共查找 1＋2×2＋4×3=17 次；

但最后一层未满，不能用 8×4，只能用 5×4=20 次，

所以 ASL＝1/12（17＋20）＝37/12≈3.08

（2）

答案：



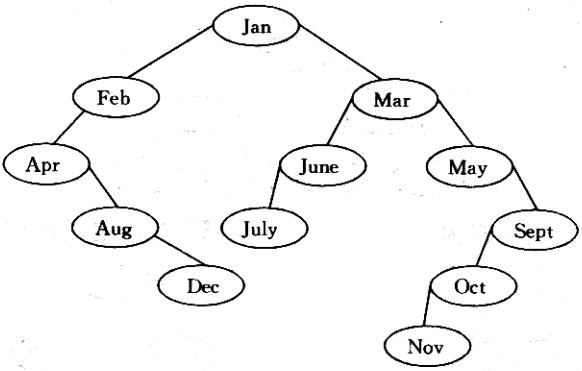
验算方法：　用中序遍历应得到排序结果：2,4,7,9,11,12,13,16,17,21

（3）

答案：

(1) 求得的二叉排序树如下图所示，在等概率情况下查找成功的平均查找长度为

$$ASL_{succ} = \frac{1}{12}(1 \times 1 + 2 \times 2 + 3 \times 3 + 4 \times 3 + 5 \times 2 + 6 \times 1) = \frac{42}{12}$$



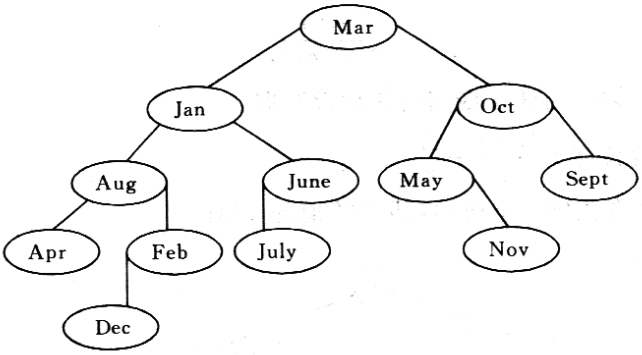
(2) 经排序后的表及在折半查找时找到表中元素所经比较的次数对照如下：

Apr	Aug	Dec	Feb	Jan	July	June	Mar	May	Nov	Oct	Sept
3	4	2	3	4	1	3	4	2	4	3	4

等概率情况下查找成功时的平均查找长度为

$$ASL_{succ} = \frac{1}{12}(1 \times 1 + 2 \times 2 + 3 \times 4 + 4 \times 5) = \frac{37}{12}$$

(3) 平衡二叉树为



它在等概率情况下的平均查找长度为

$$ASL = \frac{1}{12}(1 \times 1 + 2 \times 2 + 3 \times 4 + 4 \times 4 + 5 \times 1) = \frac{38}{12}$$

(4)

答案：

①画表如下：

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
32	17	63	49					24	40	10				30	31	46	47

②查找 63,首先要与 H(63)=63%16=15 号单元内容比较，即 63 与 31 比较 ,不匹配;然后顺移，与 46,47,32,17,63 相比，一共比较了 6 次！

③查找 60,首先要与 H(60)=60%16=12 号单元内容比较，但因为 12 号单元为空（应当有空标记），所以应当只比较这一次即可。

④对于黑色数据元素，各比较 1 次；共 6 次；

对红色元素则各不相同，要统计移位的位数。“63”需要 6 次，“49”需要 3 次，“40”需要 2 次，“46”需要 3 次，“47”需要 3 次，

所以 ASL=1/11（6＋2＋3×3+6）＝23/11

（5）

答案：

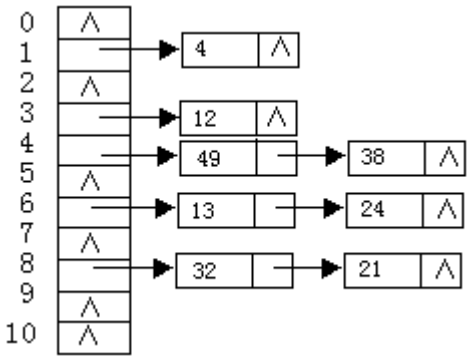
①

散列地址	0	1	2	3	4	5	6	7	8	9	10
关键字		4		12	49	38	13	24	32	21	
比较次数		1		1	1	2	1	2	1	2	

ASL_{succ} =（1+1+1+2+1+2+1+2）/8=11/8

ASL_{unsucc}=（1+2+1+8+7+6+5+4+3+2+1）/11=40/11

②



ASL_{succ} =（1*5+2*3）/8=11/8

ASL_{unsucc}=（1+2+1+2+3+1+3+1+3+1+1）/11=19/11

第 8 章　排序

1．选择题

（1）

答案：C

（2）

答案：D

（3）

答案：B

解释：对关键字进行冒泡排序，关键字逆序时比较次数最多。

（4）

答案：D

解释：比较次数最多时，第一次比较 n-1 次，第二次比较 n-2 次……最后一次比较 1 次，即(n-1)+(n-2)+…+1= n(n-1)/2。

（5）

答案：C

解释：B 选项是快速排序的最坏情况。

（6）

答案：B

解释：快速排序的平均时间复杂度为 $O(n\log_2 n)$，但在最坏情况下，即关键字基本排好序的情况下，时间复杂度为 $O(n^2)$。

（7）

答案：C

（8）

答案：D

解释：D 选项为小根堆

（9）、

答案：B

（10）

答案：C

解释：不稳定排序有希尔排序、简单选择排序、快速排序、堆排序；稳定排序有直接插入排序、折半插入排序、冒泡排序、归并排序、基数排序。

2．应用题

答案：

①直接插入排序

[2　　12]　16　30　28　10　16*　20　6　18

[2 12 16] 30 28 10 16* 20 6 18
[2 12 16 30] 28 10 16* 20 6 18
[2 12 16 28 30] 10 16* 20 6 18
[2 10 12 16 28 30] 16* 20 6 18
[2 10 12 16 16* 28 30] 20 6 18
[2 10 12 16 16* 20 28 30] 6 18
[2 6 10 12 16 16* 20 28 30] 18
[2 6 10 12 16 16* 18 20 28 30]

② 折半插入排序 排序过程同①

③ 希尔排序（增量选取 5，3，1）
10 2 16 6 18 12 16* 20 30 28 （增量选取 5）
6 2 12 10 18 16 16* 20 30 28 （增量选取 3）
2 6 10 12 16 16* 18 20 28 30 （增量选取 1）

④ 冒泡排序
2 12 16 28 10 16* 20 6 18 [30]
2 12 16 10 16* 20 6 18 [28 30]
2 12 10 16 16* 6 18 [20 28 30]
2 10 12 16 6 16* [18 20 28 30]
2 10 12 6 16 [16* 18 20 28 30]
2 10 6 12 [16 16* 18 20 28 30]
2 6 10 [12 16 16* 18 20 28 30]
2 6 10 12 16 16* 18 20 28 30]

⑤ 快速排序
12 [6 2 10] **12** [28 30 16* 20 16 18]
6 [2] **6** [10] 12 [28 30 16* 20 16 18]
28 2 6 10 12 [18 16 16* 20] **28** [30]
18 2 6 10 12 [16* 16] **18** [20] 28 30
16* 2 6 10 12 16* [16] 18 20 28 30
左子序列递归深度为 1，右子序列递归深度为 3

⑥ 简单选择排序
2 [12 16 30 28 10 16* 20 6 18]
2 6 [16 30 28 10 16* 20 12 18]
2 6 10 [30 28 16 16* 20 12 18]

2 6 10 12 [28 16 16* 20 30 18]

2 6 10 12 16 [28 16* 20 30 18]

2 6 10 12 16 16* [28 20 30 18]

2 6 10 12 16 16* 18 [20 30 28]

2 6 10 12 16 16* 18 20 [28 30]

2 6 10 12 16 16* 18 20 28 [30]

⑧ 二路归并排序

2 12 16 30 10 28 16 * 20 6 18

2 12 16 30 10 16* 20 28 6 18

2 10 12 16 16* 20 28 30 6 18

2 6 10 12 16 16* 18 20 28 30